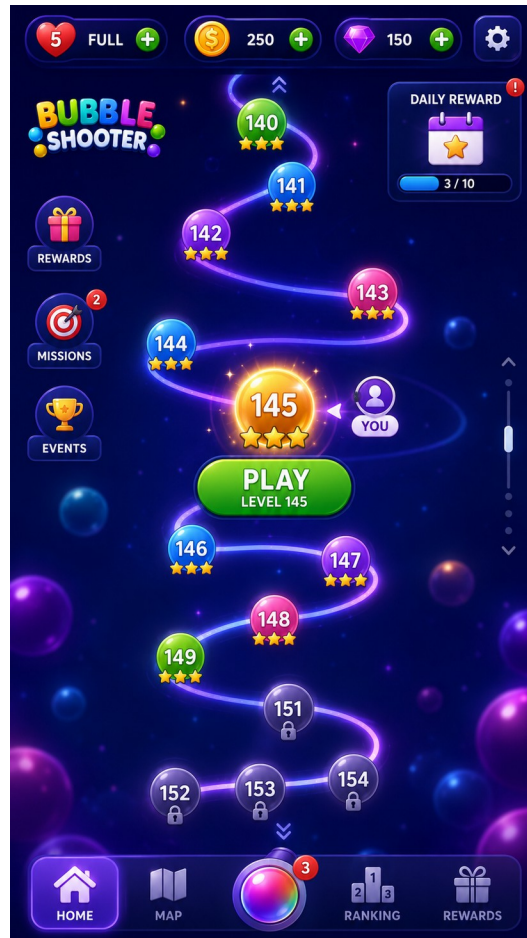


BUBBLE SHOOTER

HTML5 Game - Master Blueprint

Version 1.0 - Source of Truth



Document authority

This blueprint is the implementation authority for the Bubble Shooter project. If code, a prompt, a mockup, or a later assumption conflicts with this document, this blueprint wins until it is deliberately revised and versioned.

Project	Bubble Shooter HTML5 Game
Blueprint version	1.0
Initial level cap	10,000 addressable levels
Primary release target	Mobile-first web / HTML5
Delivery method	Phase-gated implementation with approval after every phase

Table of Contents

1. Document authority and change control
2. Product vision and non-negotiable principles
3. Approved visual direction and reference interpretation
4. Technical stack and application architecture
5. Home dashboard and infinite level map
6. Gameplay screen and asset-light rendering strategy
7. Core Bubble Shooter engine rules
8. Grid, physics, collision, snapping and resolution
9. Missions, onboarding and first 15 levels
10. Scoring, stars, shots and win/lose rules
11. Level data model, templates and deterministic generation
12. Difficulty progression and 10,000-level scalability
13. Validation, fairness and generation safety
14. State management, persistence and versioning
15. Animation, iconography, illusion and game feel
16. Performance, responsiveness, accessibility and QA
17. Initial feature status: active, visible-disabled, future
18. Phase-gated implementation roadmap
19. Codex execution protocol and completion report format
20. Definition of done and final acceptance criteria

1. Document Authority and Change Control

Source-of-truth rule

No feature should be added, removed, or materially changed because of convenience, guesswork, or a single implementation prompt. Any intentional deviation must first be recorded as a blueprint revision.

1.1 Versioning policy

- Use semantic document revisions such as v1.0, v1.1, v2.0. Minor clarifications that do not change game behavior may use a minor version; mechanic or architecture changes require a deliberate revision entry.
- Generated levels must also carry a generatorVersion so existing level IDs remain stable when future generation logic evolves.
- Every implementation phase must end with a written status report: approved, incomplete, paused, blocked, and explicitly not touched.

1.2 Conflict priority

Priority	Authority
1	Latest explicitly approved blueprint revision
2	Explicit written user/senior-dev instruction recorded after that revision
3	Approved visual reference used as direction
4	Implementation plan for the current phase
5	Existing code behavior if not contradicted above

2. Product Vision and Non-Negotiable Principles

2.1 Product vision

Build a polished, modern, mobile-first HTML5 Bubble Shooter that feels like a real game world rather than a web dashboard. The game must be asset-light but visually rich, using code-generated depth, motion, lighting, vector graphics and procedural content to create the illusion of a much larger production footprint.

2.2 Non-negotiable principles

- Game-first presentation: every screen must feel interactive, playful and alive - never like a generic SaaS dashboard.
- Asset-light by design: no mandatory full-screen background PNGs for home or gameplay; use CSS, SVG, Canvas and lightweight reusable vectors wherever practical.
- 10,000 levels at launch as addressable level IDs, with architecture that can expand far beyond 10,000 without redesigning the engine.
- First 10-15 levels teach the core mechanic through the single mission Clear All Bubbles.
- Home dashboard contains the scrollable level journey; the central Quick Play action launches the current level.
- Future tabs and feature surfaces remain visible to establish the final game shell, but they are disabled until their implementation phases are approved.
- One implementation phase at a time. Codex must not proceed beyond the explicitly approved phase.

3. Approved Visual Direction and Reference Interpretation

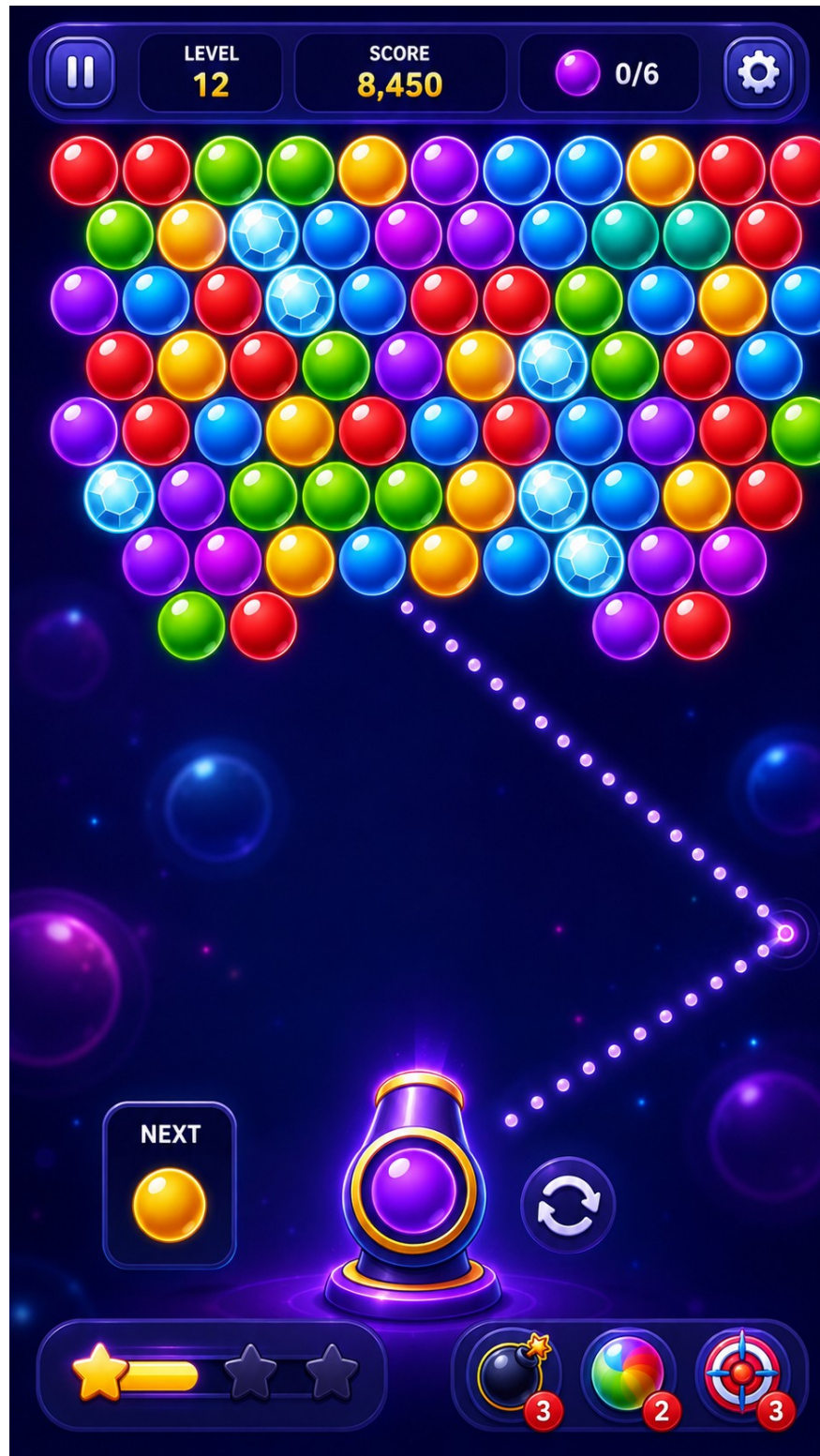
3.1 Home dashboard / scrollable level journey



Reference A - approved home-dashboard direction: fixed HUD, game-like level journey, current-level emphasis, fixed bottom navigation and a vertically scrollable map.

- This image is a visual direction reference, not a requirement to embed the image itself in the product.
- The central level-path region is the scrollable world. Top HUD and bottom navigation remain fixed to the viewport.
- The map must support continuous/virtualized vertical scrolling similar to an infinite feed, while rendering only a small window of levels around the viewport.
- The abstract navy/purple background, ambient bubbles, glows and sparkles should be recreated procedurally rather than as a large background asset.
- The reference includes future feature controls. Their presence is intentional, but their MVP interaction state is defined later in this blueprint.

3.2 Gameplay screen



Reference B - approved gameplay direction: dense readable bubble field, top HUD, dotted bank-shot trajectory, code-renderable background, shooter, next bubble and booster slots.

- Gameplay background is not a required image asset. Recreate the dark navy/purple depth with layered gradients, radial glows, blurred decorative bubbles and small particles.

- Bubbles should be rendered by the game engine with reusable visual treatment rather than individual image assets per color.
- The shooter may be a reusable SVG/vector component rendered from code. A custom raster asset is optional, not required.
- Booster buttons are part of the long-term visual layout; in the initial build they can be visible but disabled until booster phases are approved.

Visual interpretation rule

Match the approved references in mood, hierarchy, depth, color energy, spacing and game feel. Do not attempt a literal pixel clone if doing so would force heavy raster assets or harm responsiveness.

4. Technical Stack and Application Architecture

4.1 Locked baseline stack

Area	Decision	Reason
Application shell	React + TypeScript + Vite	Fast modern HTML5 delivery; strong component and build tooling.
Gameplay renderer	HTML5 Canvas 2D	Deterministic bubble rendering, collision visuals, particles and high-performance animation.
UI styling	CSS + SVG	Asset-light game UI, scalable icons and procedural backgrounds.
React motion	Motion/Framer Motion as default	Modern screen and UI transitions; use sparingly outside gameplay loop.
Icons	Lucide as primary; Iconify only when needed	Vector-first, lightweight, customizable.
Gameplay animation	Canvas-native timeline/state animation	Avoid React re-render cost for high-frequency game motion.
Persistence	localStorage initially; IndexedDB if data grows	Simple offline progress foundation; backend sync can be added later.
Testing	Unit tests + deterministic simulation tests + E2E smoke tests	Protect engine correctness and level stability.

4.2 Architecture boundaries

src/	
app/	# routing, boot, global app services
screens/	# home, gameplay, overlays
ui/	# reusable game-style UI components
game/	
engine/	# state machine + loop
grid/	# hex grid math and neighbors
physics/	# projectile movement and collision
resolver/	# match + floating bubble resolution
missions/	# mission registry and progress
levels/	# curated level data + access API
generator/	# templates, RNG, difficulty, validator
scoring/	# score, combos, star thresholds
persistence/	# progress serialization/versioning
render/	# Canvas rendering and visual effects
config/	# game-wide constants and feature flags

4.3 Separation rules

- React owns screens, menus, HUD containers and lifecycle. The gameplay engine must remain independently testable and must not depend on React for its core rules.
- Game state changes are authoritative in the engine. React observes summaries/events; it must not duplicate physics or resolver logic.
- Gameplay rendering should use a single Canvas scene or tightly controlled layered canvases. Avoid a DOM element per bubble.

- Feature flags control disabled placeholder features so future activation does not require redesigning the home shell.

5. Home Dashboard and Infinite Level Map

5.1 Screen composition

Region	Behavior
Top HUD	Fixed. Shows core currencies/resources and settings. Plus buttons may be visible but disabled until monetization/economy phases exist.
Main map viewport	Vertically scrollable/virtualized. Contains procedural background, glowing path, level nodes, current-level marker and optional contextual cards.
Current level	Strongest visual emphasis. Provides an obvious Play/Continue action.
Quick Play center button	Fixed bottom-center action. Immediately launches the current unlocked level.
Bottom navigation	Fixed. Home active; other sections remain visible but disabled until separately approved.
Future feature shortcuts	Visible where the approved layout expects them, but non-interactive or clearly disabled in MVP.

5.2 Infinite-scroll implementation

- Do not render all 10,000 level nodes simultaneously.
- Maintain a virtual world coordinate for every level but mount/render only the visible range plus a small buffer above and below the viewport.
- The winding path can be generated from deterministic control points or periodic curves so node positions are stable for a given level ID.
- When the dashboard opens, automatically position near the current level while preserving manual scroll freedom.
- Completed levels, current level and locked levels must have distinct visual states.
- No hard-coded fixed-height 10,000-node DOM page is allowed.

5.3 Initial home interaction state

Element	Initial status	Behavior
Home	ACTIVE	Shows level map and current progress.
Level nodes	ACTIVE	Completed/current nodes can open or launch the appropriate level subject to unlock rules.
Quick Play / center shooter button	ACTIVE	Launches current level immediately.
Settings	ACTIVE (basic)	At minimum sound/music/haptics toggles when those systems exist; can start as a basic panel.
Map tab	VISIBLE - DISABLED	Reserved for future dedicated map/navigation behavior if needed.
Ranking	VISIBLE - DISABLED	No leaderboard functionality yet.
Rewards	VISIBLE - DISABLED	No reward economy yet.
Missions	VISIBLE - DISABLED	No meta-mission screen yet.
Events	VISIBLE - DISABLED	No live events yet.

Element	Initial status	Behavior
Daily reward card	VISIBLE - DISABLED	Can display a coming-soon/locked state, but must not fake reward claims.
Currency + buttons	VISIBLE - MOSTLY DISABLED	Display shell values as configured; purchase/add flows are not implemented until later.

6. Gameplay Screen and Asset-Light Rendering Strategy

6.1 Gameplay layout

- Fixed/overlay top HUD for pause, level, score and mission progress.
- Bubble grid anchored to the upper gameplay region with responsive safe margins.
- Large clear aiming field between grid and shooter.
- Shooter centered near the bottom, current bubble inside/above the shooter, next bubble shown nearby.
- Booster slots remain visible in the approved layout but are disabled until booster implementation phases.
- Dotted trajectory guide may show one or more predicted wall bounces, subject to gameplay tuning.

6.2 Asset policy

Visual	Default implementation
Background	CSS/Canvas layered gradients, radial glows, ambient bubbles, particles.
Gameplay bubbles	Canvas-rendered glossy circles; optional offscreen-canvas sprite caching for performance.
Aim line	Canvas dots/segments with fade and bounce prediction.
Shooter	Reusable SVG/CSS/vector component; optional later branded asset only if necessary.
Stars	SVG or icon-library vector, styled with gradients/glow.
Common icons	Lucide/Iconify vector bases, restyled to match game UI.
Boosters	Vector icon bases + custom game-style containers; initially disabled.
Logo	May be a dedicated lightweight SVG/WebP asset if a distinctive brand mark is finalized.

No generic web-app look

Raw library icons must never be dropped into the game unchanged. Every icon needs game-specific sizing, containers, gradients, highlights, shadows, glow and interaction feedback appropriate to the visual system.

7. Core Bubble Shooter Engine Rules

7.1 Turn loop

```
AIM -> FIRE -> PROJECTILE TRAVEL -> COLLIDE -> SNAP TO HEX GRID
-> MATCH CHECK -> POP 3+ SAME-COLOR GROUP
-> DROP BUBBLES NOT CONNECTED TO CEILING
-> UPDATE SCORE + MISSION
-> CHECK WIN / LOSE
-> LOAD NEXT BUBBLE -> AIM
```

7.2 Core rules

- A valid color match is a connected group of at least 3 bubbles of the same color after the fired bubble snaps to the grid.
- Any bubble cluster no longer connected to the ceiling/top anchors after a pop must fall and be removed.
- The projectile may bounce from left and right walls. Top collision resolves into a valid top-row cell.

- Only one projectile may be active at a time.
- Input is locked during travel, snap, pop, drop and mandatory resolution states.
- Next bubble color must normally be selected from colors still present on the playable board. A color removed from the board should stop entering the queue unless a specific level rule says otherwise.
- Player wins when all mandatory level missions are satisfied.
- Player loses when the configured lose condition triggers, normally shot exhaustion with incomplete mission or danger-line breach.

8. Grid, Physics, Collision, Snapping and Resolution

8.1 Hex grid model

- Use a staggered row-offset hex arrangement. Every logical bubble has row/column coordinates and a derived world-space center point.
- Neighbor lookup must be centralized in grid math and covered by unit tests for odd/even row offsets.
- Grid occupancy is authoritative; visual positions are derived from grid state, never the reverse.

8.2 Projectile physics and collision

- Projectile movement must be frame-rate independent using delta time.
- Prevent tunneling through bubbles with swept collision checks or bounded movement substeps when speed is high.
- Wall reflection must preserve speed and invert the horizontal direction component.
- Collision resolution must stop at the earliest valid impact before snapping.

8.3 Snapping algorithm

1. Collect candidate empty grid cells around the impact region or impacted bubble.
2. Reject occupied, unreachable or out-of-bounds candidates.
3. Choose the nearest valid cell using deterministic tie-breaking.
4. Commit the bubble to grid occupancy before running match detection.
5. Assert that no two logical bubbles occupy the same cell and no bubble remains between cells.

8.4 Match and floating-bubble resolver

- Use BFS/DFS flood fill from the newly placed bubble to find the same-color connected component.
- If size ≥ 3 , mark the entire group for pop; otherwise retain the placed bubble.
- After pop removal, flood-fill from all ceiling-connected bubbles. Any remaining bubble not reached by this connectivity pass is floating and must fall.
- Score and mission events are emitted once per resolved bubble to prevent double counting.

9. Missions, Onboarding and the First 15 Levels

9.1 Initial onboarding rule

Locked onboarding decision

Levels 1 through 15 use one mission only: CLEAR ALL BUBBLES. The purpose is to teach aiming, wall bounces, matching, dropping and board reading without extra objective complexity.

9.2 First 15 progression

Levels	Mission	Colors	Design focus	Pressure
1-3	Clear All Bubbles	3	Basic aiming, direct matches, obvious clusters	Very generous shots; no miss penalty
4-5	Clear All Bubbles	3	Introduce simple ceiling supports and drop opportunities	Generous shots
6-10	Clear All Bubbles	3-4	Larger shapes and controlled bank-shot opportunities	Moderate but forgiving
11-15	Clear All Bubbles	4	Split clusters, hanging structures, more planning	Slightly tighter but fair

9.3 Mission registry after onboarding

Mission type	Definition	Initial availability
CLEAR_ALL	Remove all required bubbles from the board.	Levels 1+
POP_COLOR	Pop/drop a target count of one or more colors.	Introduce after level 15
DROP_BUBBLES	Disconnect and drop a target count.	Introduce after level 15
CLEAR_MARKED	Clear bubbles carrying a code-rendered marker/ring.	Introduce after fundamentals are stable
REACH_SCORE	Reach a configured score threshold.	Use sparingly after onboarding
MULTI_OBJECTIVE	Combine at most two mandatory objectives initially.	Later difficulty bands only

9.4 Future mission extensibility

- The mission system must use a registry/interface so new mission types can be added without rewriting the turn loop.
- Examples for future phases: rescue target, frozen bubbles, stone bubbles, anchors, special collection. These are not part of the initial engine scope.
- Mission UI must derive from mission descriptors rather than hard-coded level-specific components.

10. Scoring, Stars, Shots and Win/Lose Rules

10.1 Baseline score model

Event	Baseline behavior
Matched bubble	Base points per bubble
Dropped bubble	Higher base value than a normal pop to reward strategic disconnection
Large match	Size bonus above the minimum match
Combo/cascade	Multiplier or additive bonus
Bank shot	Optional bonus, subject to tuning
Remaining shots	Optional end-of-level completion bonus

10.2 Star rating

- Completing the mission unlocks at least 1 star.
- 2-star and 3-star thresholds are level-specific score targets derived from curated data or generator difficulty estimates.
- Stars are performance ratings, not separate mandatory missions.
- Persist best score and highest star count per level.

10.3 Shot system

- Each fired bubble consumes one shot unless a future special rule explicitly says otherwise.
- Levels 1-15 use forgiving shot budgets.
- If shots reach zero while missions remain incomplete, the level ends in loss unless a future continue system is explicitly enabled.
- A traditional miss-counter/new-row pressure mechanic may be added to selected later templates; it is disabled for the first 15 levels and must not be globally forced on every level.

10.4 Danger line

- The danger line is a configurable fail condition, not an always-visible requirement for every early level.
- If enabled, any occupied bubble crossing the configured danger threshold after resolution triggers loss.

11. Level Data Model, Templates and Deterministic Generation

11.1 Level access API

```
getLevel(levelId: number): LevelConfig
```

1-15 -> hand-curated onboarding levels

16-100 -> controlled curated/template-assisted levels

101+ -> deterministic template-driven generation, with overrides allowed

11.2 Level model

```
interface LevelConfig {  
  id: number;  
  generatorVersion: number;  
  seed: string;  
  board: BoardDefinition;  
  colors: BubbleColor[];  
  shotLimit: number;  
  dangerLine?: DangerLineConfig;  
  missPenalty?: MissPenaltyConfig;  
  missions: MissionConfig[];  
  starThresholds: [number, number, number];  
  difficulty: DifficultyDescriptor;  
  templateId: string;  
}
```

11.3 Template library

- Initial reusable templates may include triangle, diamond, wave, columns, split clusters, hanging clusters, tunnel, wide top, islands and zigzag.
- Templates describe shape/occupancy structure, not final colors or mission targets.
- Templates must support parameter ranges so one template can produce many distinct but recognizable levels.

11.4 Deterministic generation

```
levelId + generatorVersion  
  -> seeded PRNG  
  -> difficulty band  
  -> template + parameters  
  -> color distribution  
  -> board generation  
  -> mission selection  
  -> shot/star targets  
  -> validation  
  -> stable LevelConfig
```

- The same level ID and generatorVersion must produce the same result on every device.
- Generation must not depend on wall-clock time, browser randomness or runtime order.
- If validation fails, use deterministic retry seeds such as `\${levelId}:retry:\${attempt}`.

12. Difficulty Progression and 10,000-Level Scalability

12.1 Difficulty waves

- Difficulty must rise in waves rather than a single monotonic slope.
- Use recovery levels after hard/challenge levels to avoid fatigue.
- Control difficulty through color count, cluster structure, board height, connectivity, shot budget, mission complexity and optional pressure systems.

Band	Typical color count	Typical content
Levels 1-10	3-4	Onboarding, direct matches, high clarity
11-50	4	More shape complexity and bank shots
51-200	4-5	Broader mission variety and template combinations
201+	5-6	Deeper planning, multi-objective and advanced templates where approved

12.2 Scaling beyond 10,000

- The initial public level cap is a configuration value, not an architectural hard limit.
- Increasing the cap should primarily involve content/difficulty configuration, new templates and optional new mission types - not a rewrite of the engine.
- Progress storage must not allocate a giant fixed object for every possible level. Store only played/completed level records plus current progression metadata.

13. Validation, Fairness and Generation Safety

13.1 Required validation checks

- Board contains at least one playable bubble and respects template bounds.
- Mission targets reference colors/items that actually exist in sufficient quantity.
- Generated target counts cannot exceed what the board can provide under the mission counting rules.
- Shot limits remain within configured difficulty ranges and are not trivially impossible by simple heuristics.
- Initial board does not already violate the danger line unless deliberately authored.
- Queue colors are compatible with remaining board colors.
- No overlapping cells, invalid coordinates or disconnected serialization errors.

13.2 Solvability honesty

Important limitation

Static validation can reject obviously invalid or unfair levels, but it does not mathematically prove every generated board is solvable. A later bot/simulation gate can test candidate levels more deeply. Until then, use conservative generation rules, telemetry/manual review and deterministic overrides for any problematic level IDs.

14. State Management, Persistence and Versioning

14.1 Persisted player progress

```
PlayerProgress {  
  schemaVersion  
  currentLevel  
  completedLevels: { [levelId]: { stars, bestScore } }  
  settings: { music, sound, haptics }  
  currencies?: future  
}
```

14.2 Persistence rules

- Save progress after level completion and important settings changes.
- Never overwrite a higher star rating or better score with a lower result.
- Use a schemaVersion and migration path so future releases can safely evolve saved data.
- Generated level identity must include generatorVersion to prevent silent content drift for existing players.
- Backend/cloud sync is a later phase and must not block the first hosted build.

15. Animation, Iconography, Illusion and Game Feel

15.1 Experience target

The product should feel like more than a functional game. It should create the illusion of depth, energy and a living world through code-driven motion and layered effects, without depending on a large asset library.

15.2 Approved illusion techniques

- Layered parallax: distant glows/bubbles move less than foreground elements during map scroll.
- Pseudo-3D surfaces: gradients, highlights, inner shadows, soft cast shadows and perspective scaling.
- Ambient motion: slow drifting bubbles, subtle particles and light pulses that never distract from gameplay.
- Depth-of-field illusion: larger blurred foreground bubbles and smaller dim background bubbles.
- Micro camera response: restrained impact pulse or screen nudge on major pops/drops; avoid excessive shake.
- Path energy: animated light travel or subtle shimmer on the level route.
- Current-level emphasis: glow breathing, sparkles and a small positional lift, without making the screen visually noisy.
- Shooter feel: rotation, anticipation, recoil, muzzle/energy pulse and settle-back motion.
- Transitions: screens should feel connected through motion, not like abrupt web-page navigation.

15.3 Animation technology rules

Context	Preferred method
Gameplay bubbles/projectiles/particles	Canvas-native animations tied to engine time
React UI transitions	Motion/Framer Motion
Complex one-off UI sequences	GSAP only if justified and approved
Icons/path effects	SVG/CSS animation
Reduced motion	Respect user preference by simplifying non-essential ambient

Context	Preferred method
	effects

15.4 Motion quality rules

- Use modern easing/spring behavior; avoid default linear or generic browser transitions.
- Animations must communicate state or add depth, not merely decorate every element.
- Gameplay-critical feedback must remain readable at reduced motion settings.
- No animation should block input longer than the engine state actually requires.

16. Performance, Responsiveness, Accessibility and QA

16.1 Performance targets

- Target smooth 60 FPS on modern mobile devices; degrade non-essential particles/glows before compromising core gameplay responsiveness.
- Use devicePixelRatio caps for Canvas to prevent excessive fill-rate on very high-density screens.
- Pre-render/cache reusable bubble visual layers or offscreen sprites when beneficial.
- Virtualize the level map and recycle nodes rather than mounting thousands of elements.
- Pause or throttle ambient animations when the document is hidden or the game is paused.

16.2 Responsive behavior

- Portrait mobile is the primary design target.
- Desktop/tablet may letterbox or center a portrait playfield while preserving touch/mouse control.
- Safe-area insets must be respected on notched devices.
- Gameplay geometry must derive from the measured playfield, not hard-coded screenshot pixels.

16.3 Accessibility and input

- Support touch, mouse and pointer input through one normalized input layer.
- Use adequate contrast for mission/HUD text and distinguish bubble colors with optional secondary cues if future testing shows color-only confusion.
- Provide sound/music toggles and a reduced-motion-friendly mode.
- Keyboard aiming is optional for the first build but architecture should not prevent later support.

16.4 QA requirements

- Unit tests for hex neighbors, seeded RNG, mission counters, score logic and persistence migrations.
- Deterministic tests for selected level IDs: same input/version -> same generated LevelConfig.
- Physics regression tests for wall bounces, collision edges and snap candidates.
- E2E smoke path: launch app -> home -> start level -> shoot -> complete/lose -> return home -> progress persists.
- Performance profiling on at least one representative mobile-sized viewport before first hosting approval.

17. Initial Feature Status: Active, Visible-Disabled and Future

Feature	Status at initial implementation	Notes
Home dashboard + scrollable level map	ACTIVE	Core navigation surface.
Quick Play / Continue	ACTIVE	Launches current level.
Level selection/unlock progression	ACTIVE	Supports 10,000 IDs; map virtualized.
Core Bubble Shooter gameplay	ACTIVE	Aim, shoot, bounce, snap, match, pop, drop.
Clear All mission	ACTIVE	Levels 1-15 and reusable later.
Additional mission engine	FOUNDATION ACTIVE	Introduce mission types after onboarding according to phase approval.
Stars and best score	ACTIVE	Persist per level.
Basic settings	ACTIVE/FOUNDATION	At least audio-related toggles as systems exist.
Boosters	VISIBLE - DISABLED	Slots/icons may appear; no effect until implemented.
Ranking	VISIBLE - DISABLED	Future leaderboard feature.
Rewards/Daily Reward	VISIBLE - DISABLED	Future reward economy.
Missions meta-screen	VISIBLE - DISABLED	Separate from per-level objectives; future feature.
Events	VISIBLE - DISABLED	Future live/rotating content.
Shop	VISIBLE - DISABLED if shown	Future economy/monetization.
Cloud accounts/sync	FUTURE	Not required for first hosted build.
Ads/IAP	FUTURE	Not part of core engine phase.
Advanced blockers/special bubbles	FUTURE	Add only after core engine review.

Disabled means disabled

Visible future controls must not navigate to empty broken screens, mutate currency, or pretend to grant rewards. Use disabled styling and optional lightweight “Coming Soon” feedback only if explicitly approved.

18. Phase-Gated Implementation Roadmap

Phase	Name	Scope
Phase 0	Blueprint + project audit	Create/confirm this source-of-truth document and inspect any existing codebase. No gameplay implementation beyond necessary audit notes.
Phase 1	Project foundation	React/TS/Vite shell, directory architecture, game loop skeleton, feature flags, seeded RNG utility, test setup.
Phase 2	Hex grid system	Grid coordinates, world mapping, neighbor math, occupancy, debug visualization, unit tests.
Phase 3	Shooter and aiming	Pointer/touch aiming, angle limits, current/next bubble, trajectory visualization, input lock.
Phase 4	Projectile physics	Frame-independent motion, wall bounce, collision detection, anti-tunneling strategy.
Phase 5	Snap system	Deterministic nearest-valid-cell snapping and occupancy integrity.
Phase 6	Match resolver	3+ same-color connected match detection

Phase	Name	Scope
		and pop events.
Phase 7	Floating bubble resolver	Ceiling connectivity, falling bubbles, drop scoring.
Phase 8	State machine + turn lifecycle	Strict engine states, timing, input locks, pause handling.
Phase 9	Mission 1 + first 15 levels	Clear All mission, 15 curated onboarding levels, generous early tuning.
Phase 10	Score, stars, save progress	Scoring, thresholds, completion records, persistence and migrations.
Phase 11	Level access + templates	Unified getLevel API, template definitions, level metadata.
Phase 12	Mission registry expansion	Pop Color, Drop Bubbles, Clear Marked, Reach Score, multi-objective support.
Phase 13	Deterministic generator + validator	101+ generation pipeline, retry strategy, fairness validation, generatorVersion.
Phase 14	Home dashboard/map	Approved visual direction, virtualized infinite scroll, current-level focus, Quick Play, disabled future tabs.
Phase 15	Gameplay visual integration	Procedural background, glossy Canvas bubbles, vector shooter, aim effects, disabled boosters.
Phase 16	Animation/illusion polish	Parallax, ambient motion, transitions, recoil, particles, reduced-motion behavior.
Phase 17	QA + first hosted build	Cross-device testing, performance profiling, regression fixes, deployment candidate.
Phase 18	Senior-dev review gate	Collect feedback. Only after approval plan boosters, blockers, events, economy or other advanced systems.

19. Codex Execution Protocol and Completion Report Format

19.1 Mandatory prompt rule

Every Codex implementation prompt must end with

“Implement this phase only. Do not proceed to the next phase. Stop after tests and the completion report.”

19.2 Required working sequence

1. Read the latest blueprint and relevant existing files before editing.
2. State the intended scope and identify anything explicitly out of scope.
3. Implement only the current phase.
4. Run the relevant tests, lint/type checks and any focused runtime checks.
5. Produce a completion report.
6. Wait for approval or fix instructions before touching the next phase.

19.3 Completion report template

```
PHASE: <number + name>
STATUS: COMPLETE / PARTIAL / BLOCKED

FILES CREATED:
- ...

FILES MODIFIED:
- ...

IMPLEMENTED:
- ...

NOT TOUCHED:
- ...

TESTS / CHECKS RUN:
- ...

KNOWN ISSUES OR RISKS:
- ...

APPROVAL NEEDED BEFORE NEXT PHASE:
- Yes
```

19.4 Change discipline

- No opportunistic refactors outside current phase unless required to keep the build working; any such change must be explicitly reported.
- No silent library changes. New dependencies require a stated reason and should be minimized.
- No placeholder feature may become active merely because its UI exists.

- No generated asset should be introduced when the blueprint calls for code-generated visuals unless explicitly approved.

20. Definition of Done and Final Acceptance Criteria

20.1 Core engine acceptance

- Accurate staggered hex grid and neighbors.
- Reliable touch/mouse aiming and wall bounces.
- No projectile tunneling or overlapping snapped bubbles under tested conditions.
- Correct match-3+, pop and ceiling-disconnect drop behavior.
- Strict turn-state input locking with no double shots during resolution.
- Correct next-bubble color pruning based on remaining board colors.
- Clear win/loss rules and persistent progress.

20.2 Content acceptance

- Levels 1-15 are curated and all use Clear All Bubbles.
- Additional mission types are registry-driven and can be enabled after onboarding.
- 10,000 level IDs are addressable without creating 10,000 hand-written files.
- Level generation is deterministic and versioned.
- Problematic generated level IDs can be overridden without breaking the system.

20.3 UI/experience acceptance

- Home screen feels like a game world and matches the approved visual direction in hierarchy and mood.
- Level map scrolls smoothly and uses virtualization rather than rendering all levels.
- Top HUD and bottom navigation remain fixed while the map scrolls.
- Quick Play launches the current level.
- Future tabs are visible but genuinely disabled until their implementation phases.
- Gameplay and home backgrounds are created procedurally/code-first, not dependent on large full-screen background image assets.
- Modern motion, vector icons and depth illusions are present without sacrificing mobile performance.

20.4 First hosted build exit gate

Release candidate condition

The first hosted build is ready for senior-dev review only when the core engine, first 15 onboarding levels, scalable level access/generation foundation, scrollable home dashboard, progress persistence and approved asset-light visual direction all work together without known critical gameplay-state bugs.

END OF MASTER BLUEPRINT v1.0